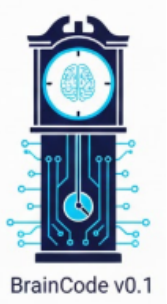
BrainCode v0.1



© 2015-2025 by Daniel Müller

A Hybrid Neuro-Temporal Language **2**
Host Language Integration 2
I. Syntax and Core Components **2**
1. Variables and Functions (Host Language) 2
2. Time Access 2
II. BrainCode Parallel Syntax **2**
1. Simultaneous Assignment 2
2. Conditional Assignment Block 3
3. Action Calls (Side Effects) 3
III. Programming Examples **3**
1. Hybrid Counter with Event Logging 3
2. Fibonacci Sequence with Initialization 4
Time-Value Boards: Capturing the Temporal State **4**
The Structure of a Time-Value Board **4**
1. The Time Axis (Rows) 5
2. The Variable Axis (Columns) 5
Operational Role in BrainCode **5**
1. Enabling Past Access 5
2. The Flow of Computation 6
Applications and Benefits **6**
1. Advanced Debugging and Verification 6
2. Post-Hoc Analysis 6
Time-Value Board (TVB) for the Fibonacci Sequence 7
Explanation of the Process 8
PLY Definition **9**
1. Lexical Analysis (Tokens) 9
2. Grammar (Parsing Rules) 10
Key BrainCode Structures in the Grammar: 12

Abstract

BrainCode is presented here as an **embedded Domain Specific Language (DSL)** designed to run concurrently within a standard sequential programming environment (e.g., Python). It defines a computation graph of **simultaneous assignments** that execute synchronously at every time step.

A Hybrid Neuro-Temporal Language

BrainCode is presented here as an **embedded Domain Specific Language (DSL)** designed to run concurrently within a standard sequential programming environment (e.g., Python). It defines a computation graph of **simultaneous assignments** that execute synchronously at every time step.

Host Language Integration

The host language (Python) handles sequential tasks, initial setup, and external I/O. BrainCode assignments and calls are **initialized** sequentially but executed **in parallel** during the host language's runtime loop (the simulation clock).

I. Syntax and Core Components

1. Variables and Functions (Host Language)

Standard host language syntax is used for initial setup. All variables and functions defined here are **static constants** or **sequential setup logic** that feeds into the BrainCode graph.

```
Python

# Standard Python code for initialization
WEIGHT_A = 0.5
TIME_LIMIT = 50

def complex_filter(data_in):
    # Standard sequential function
    return data_in * 2

```

2. Time Access

The system provides a built-in, read-only variable for the current time step.

Syntax	Meaning
~T	The current global time step counter (Tick).
var~N	The value of var from **N time steps in the past**.

II. BrainCode Parallel Syntax

The following elements define the concurrent, temporal computation graph.

1. Simultaneous Assignment

Assignments define a variable's evolution rule for the next time step. The variable name must be enclosed in brackets: **:[Variable] := Expression**.

```
Python

# The new position is defined by the old position (pos~1) plus the old velocity (vel~1).
:pos := pos~1 + vel~1
```

2. Conditional Assignment Block

This structure allows a block of parallel actions (assignments or calls) to execute only when a temporal condition is met. The expression is the **condition for execution**.

Structure	Description
?(Expression): [Assignments or Calls]	Executes the block of parallel assignments and/or calls if Expression evaluates to true.

The block is defined by indentation, similar to a standard if statement.

```
Python

# This entire block executes in parallel only when the counter reaches 100
?(counter~1 == 100):
    :flag_done := True
    !(print("Limit Reached!"))
```

3. Action Calls (Side Effects)

Actions are calls that produce **side effects** (like printing, file I/O). They are always preceded by ! and must be contained within a **conditional assignment block** to execute.

Structure	Description
!(call_name(arguments))	Executes the side effect call.

III. Programming Examples

1. Hybrid Counter with Event Logging

This example shows a counter defined in BrainCode and a side effect triggered by the tick count.

```
Python
# Initial Setup (Host Environment)
# (Assume 'start_value' is defined here)
```

```
# BrainCode Graph Definition (Executed at every tick)
# 1. Simple Counter Assignment
:counter := counter~1 + 1

# 2. Conditional Block: Check tick count for logging
?(~T % 5 == 0):
    !(print(f"[{~T}] Counter value: {counter~1}"))
    :log_flag := True

# 3. Conditional Block: Reset the counter after a limit
?(counter~1 > 1000):
    :counter := 0
    !(print("Counter reset triggered."))
```

2. Fibonacci Sequence with Initialization

This demonstrates handling initial conditions using the time operator ~T.

```
Python
# 1. Fibonacci Sequence Assignment
# Uses the ~T value to set initial conditions (T=0 and T=1).
:fib := 0 if ~T == 0 \
        else 1 if ~T == 1 \
        else fib~1 + fib~2

# 2. Conditional Block: Stop and print final result after 10 ticks
?(~T == 10):
    !(print(f"Fibonacci at T=10 is: {fib~1}"))
    !(stop_simulation())
```

Time-Value Boards: Capturing the Temporal State

The **Time-Value Board (TVB)** is a critical conceptual and implementation component for a Hybrid Neuro-Temporal Language like BrainCode v0.1. Given that BrainCode defines a computation graph of **simultaneous assignments** that execute **synchronously at every time step ($\sim T$)**, the TVB serves as the complete memory ledger that records the value of *every* parallel variable for *every* tick of the simulation clock.

In essence, a TVB transforms a dynamic, concurrent system execution into a static, queryable dataset, which is indispensable for debugging, logging, and post-hoc analysis.

The Structure of a Time-Value Board

Conceptually, the TVB can be visualized as a **two-dimensional matrix** or a database table where the rows represent the **time steps ($\sim T$)** and the columns represent the **parallel variables** defined in the BrainCode graph.

Variable/Time	~T=0	~T=1	~T=2	...	~T=N
counter	0	1	2	...	N
pos	10.0	10.5	11.5	...	V_N
fib	0	1	1	...	F_N
log_flag	False	False	True (if $2\%5=0$)	...	False

1. The Time Axis (Rows)

Each row in the board corresponds to a single, synchronous execution cycle—a "tick"—of the host language's simulation loop. The index of this row is the value of the built-in temporal variable **$\sim T$**.

- **Synchronization:** Since all BrainCode assignments are simultaneous, the values recorded for any given $\sim T$ are fully **consistent** and represent the final state of the parallel graph for that specific moment.

2. The Variable Axis (Columns)

Each column tracks a **single, specific parallel variable** (e.g., :counter, :pos, :fib).

- **Completeness:** The TVB must log the value of *every* variable defined by a simultaneous assignment (:[Variable] := ...), regardless of whether that value changed from the previous step. This guarantees that the historical record is complete and continuous.
- **Data Types:** The columns store the native data type of the variable (e.g., integer for counter, float for pos, boolean for log_flag).

Operational Role in BrainCode

The TVB is the underlying mechanism that makes the core temporal syntax of BrainCode possible: **Past Access ($\text{var}\sim N$)**.

1. Enabling Past Access

When a BrainCode expression uses the past access operator, such as **fib~2** (the value of fib two steps in the past), the runtime does not need to compute this value; it simply performs a **lookup** in the Time-Value Board.

$$:fib := \text{fib}\sim 1 + \text{fib}\sim 2$$
When calculating the new value of :fib at time T, the system retrieves:

- $\text{fib}\sim 1$ from the TVB at time row **$T-1$**.
- $\text{fib}\sim 2$ from the TVB at time row **$T-2$**.

2. The Flow of Computation

1. **Read Phase:** At the start of time step $\sim T$, the system reads all required past values ($\text{var}\sim N$) from the already-populated rows ($< T$) of the TVB.
2. **Execution Phase:** The parallel computation graph runs, evaluating all simultaneous assignments to determine the new values for all variables at time T.
3. **Write Phase:** Once all new values for time T are computed, they are written **synchronously** into the new row for $\sim T$ in the TVB. This new row then becomes the available "past" data ($\text{var}\sim 1$) for the next step, $\sim T+1$.

Applications and Benefits

The persistence and queryability provided by the TVB are crucial for developing and verifying complex temporal systems.

1. Advanced Debugging and Verification

Because every intermediate value is stored, developers can step backward in time to inspect the exact state of a computation, eliminating the guesswork often associated with debugging concurrent or time-sensitive logic.

- **Traceability:** Trace the dependency chain of a variable's value across multiple time steps. If pos is wrong at $T=100$, one can instantly check the values of pos~1 and vel~1 that produced it.
- **Replay:** The entire simulation can be perfectly replayed simply by reading the values from the TVB, allowing for consistent, reproducible bug fixing.

2. Post-Hoc Analysis

The TVB becomes the definitive data source for analyzing the simulation's performance and behavior.

- **Visualisation:** The columns can be easily plotted against the time axis ($\sim T$) to generate continuous graphs of any variable's evolution (e.g., a time-series plot of the counter or fib values).
- **Statistical Analysis:** Complex metrics that depend on the entire history of a variable (e.g., maximum velocity reached, average flag state) can be calculated *after* the simulation has completed, independent of the host language's main sequential loop.

Would you like to see an example of how the TVB would be populated for the Fibonacci sequence example you provided?

Time-Value Board (TVB) for the Fibonacci Sequence

The TVB stores the value of :fib for the current time step ($\sim T$) in each row.

~T (Time Step)	fib~2 (Value at T−2)	fib~1 (Value at T−1)	Calculation for fib at T	New Value fib (Result)
0	N/A	N/A	0 (Initialization: $\sim T == 0$)	**0**
1	N/A	0	1 (Initialization: $\sim T == 1$)	**1**
2	0	1	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 1 + 0$	**1**
3	1	1	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 1 + 1$	**2**
4	1	2	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 2 + 1$	**3**
5	2	3	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 3 + 2$	**5**
6	3	5	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 5 + 3$	**8**

7	5	8	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 8 + 5$	**13**
8	8	13	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 13 + 8$	**21**
9	13	21	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 21 + 13$	**34**
10	21	34	$\text{fib}\sim 1 + \text{fib}\sim 2 \implies 34 + 21$	**55**

Explanation of the Process

1. **Time $\sim T=0$ and $\sim T=1$ (Initialization):**
    - The if conditions use $\sim T$ to directly assign the starting values of 0 and 1. No access to $\text{fib}\sim 1$ or $\text{fib}\sim 2$ is needed in these steps.
2. **Time $\sim T=2$ (First Historical Access):**
    - The calculation requires $\text{fib}\sim 1$ and $\text{fib}\sim 2$.
    - The system looks up in the TVB:
        - $\text{fib}\sim 1$ (Value from $T=1$): **1**
        - $\text{fib}\sim 2$ (Value from $T=0$): **0**
    - Calculation: $1 + 0 = 1$. The value **1** is written into the $T=2$ row of the TVB.
3. **Time $\sim T=3$:**
    - The system looks up in the TVB:
        - $\text{fib}\sim 1$ (Value from $T=2$): **1**
        - $\text{fib}\sim 2$ (Value from $T=1$): **1**
    - Calculation: $1 + 1 = 2$. The value **2** is written into the $T=3$ row of the TVB.

This example clearly demonstrates how the TVB provides the **historical values** ($\text{fib}\sim 1$ and $\text{fib}\sim 2$) for the **current calculation** (:fib), operationalizing the temporal dependency that is core to the BrainCode language.

Would you like to see the TVB populated for the Hybrid Counter example, which involves multiple variables and conditional assignments?

PLY Definition

This definition assumes the BrainCode syntax is embedded within a Python environment, where the parser only handles the specific parallel BrainCode structures.

1. Lexical Analysis (Tokens)

The tokens define the basic building blocks (keywords, symbols, and data types) of the language.

Token Name	Regular Expression (Regex)	Description
TIME_ACCESS	~T	The current global time step counter.
PAST_OPERATOR	~	The operator for past variable access (e.g., var~N).
SIM_ASSIGN_START	:[	Start of a simultaneous assignment.
SIM_ASSIGN_END	]:	End of a simultaneous assignment.
ASSIGN	:=	The simultaneous assignment operator.
COND_START	?\(	Start of a conditional assignment block.
COND_END	\):	End of a conditional assignment block.

ACTION_CALL	!	The side-effect action call operator.
ID	[a-zA-Z_][a-zA-Z_0-9]*	Variable names (e.g., pos, counter).
NUMBER	\d+	Integer values.
LPAREN	\(	Left parenthesis.
RPAREN	\)	Right parenthesis.
PLUS, MINUS, TIMES, DIVIDE	\+, -, *, /	Arithmetic operators.
EQ, NEQ, LT, GT, LE, GE, MOD	==, !=, <, >, <=, >=, %	Comparison and modulo operators.
AND, OR	and, or	Logical operators (assuming Python keywords).
NEWLINE	\n+	Line separator (important for indentation).
INDENT, DEDENT	(Handled by a separate routine)	Block structure (similar to Python).

Note on Tokens: Standard Python operators and function calls within the Expression part would typically be handled by a sub-lexer or embedded Python parsing, but they are included here for completeness of the BrainCode structures.

---

2. Grammar (Parsing Rules)

The grammar defines the structure of valid BrainCode programs. The core of the language is the **BrainCode Graph Definition** (sim_program), which is a sequence of parallel assignment and conditional blocks.

Python

```
# The overall BrainCode program embedded within the Python host

# (This is the section executed at every time step)
sim_program : sim_statement
            | sim_program sim_statement

# A simultaneous statement is either a simple assignment or a conditional block
sim_statement : simultaneous_assignment
              | conditional_block
              | NEWLINE

# 1. Simultaneous Assignment Rule

# Example: :[Variable] := Expression
simultaneous_assignment : SIM_ASSIGN_START ID SIM_ASSIGN_END ASSIGN expression
NEWLINE

# 2. Conditional Assignment Block Rule

# Example: ?(Expression): [Block]
conditional_block : COND_START expression COND_END NEWLINE INDENT sim_block
DEDENT

# A block is a sequence of parallel actions/assignments inside a conditional block
sim_block : sim_block sim_action
          | sim_action

# A parallel action inside a conditional block can be an assignment or a side-effect call
sim_action : simultaneous_assignment
           | action_call

# 3. Action Call (Side Effect) Rule

# Example: !(call_name(arguments))
action_call : ACTION_CALL function_call NEWLINE

# Time-Access Variable Rule (used in Expressions)
# Example: var~N
time_variable : ID PAST_OPERATOR NUMBER
              | TIME_ACCESS # For ~T



# --- EXPRESSION RULES —
# An expression can be a combination of terms, variables, numbers, and time-access.
# This part handles the arithmetic, logical, and temporal operations.
expression : term
           | expression PLUS term
           | expression MINUS term
           | expression EQ term # ==
```

```
        | expression GT term # >
        | expression LT term # <

        # ... (other comparison/logical operators)

term : factor
    | term TIMES factor
    | term DIVIDE factor
    | term MOD factor

factor : NUMBER
       | ID
       | time_variable
       | function_call
       | LPAREN expression RPAREN



# A simple function call structure (e.g., print(f"..."))

function_call : ID LPAREN argument_list RPAREN

argument_list : expression
              | argument_list ',' expression
              | # empty
```

Key BrainCode Structures in the Grammar:

- **Temporal Logic:** The rules for time_variable directly incorporate the core temporal syntax: TIME_ACCESS (~T) and the PAST_OPERATOR (ID PAST_OPERATOR NUMBER, e.g., fib~2).
- **Parallelism:** The simultaneous_assignment rule is clearly delineated by the SIM_ASSIGN_START (:[) and SIM_ASSIGN_END (]:) delimiters.
- **Control Flow:** The conditional_block rule is marked by COND_START (?() and uses INDENT/DEDENT to group the parallel actions within the condition, mirroring Python's block structure.
- **Side Effects:** The ACTION_CALL (!) token enforces that side-effect functions must be part of a block structure (sim_block).